

Tabling and Mode-Driven Goal Scheduling

The Termination Paper

L.J. Brian Cowan
loveJesus@loveJes.us

Draft - February 2, 2026

Abstract

We present a tabling system that guarantees termination for finite-domain relational queries through mode-driven goal scheduling. The key insight is that *goal reordering by groundness* transforms potentially non-terminating left-to-right evaluation into terminating schedules.

Given a conjunction $(g_1, g_2, \dots, g_n)$, we dynamically reorder goals so that those with more ground arguments execute first. This ensures that generators (which enumerate infinite streams) are deferred until constraining goals have bound their shared variables.

For mutually recursive predicates, we employ SLG resolution with strongly connected component (SCC) detection. Goals within an SCC must complete together; Tarjan’s algorithm identifies these groups in $O(V + E)$ time.

Our implementation demonstrates:

- **Termination:** Composition queries that would loop without tabling now terminate with finite answer sets
- **Optimal scheduling:** Mode analysis achieves automatic goal reordering equivalent to hand-tuned Prolog
- **Tensor construction:** Each solved query populates a sparse Boolean tensor incrementally, bridging streams and matrices

We prove that for FD-miniKanren with bounded term depth, tabling with mode analysis guarantees termination while preserving completeness.

1 Introduction

Scope Clarification. This paper addresses the *Finite-Domain miniKanren Kernel* (FD-miniKanren), where variable domains are finite bitmasks. Standard miniKanren unification over unbounded term structures uses a separate reference implementation; the bit-parallel approach accelerates the constraint propagation core, not full structural unification.

Consider the query:

```
(run* (A B X C)
      (appendo A B X)
      (appendo X C [0 1 2])))
```

Evaluated left-to-right, this query diverges. The first goal (`appendo A B X`) generates an infinite stream of values for X , never reaching the second goal that would constrain it.

Yet the query has exactly 10 solutions. How can we find them?

The answer lies in *mode-driven goal reordering*. By analyzing which arguments are ground (known) versus free (unknown), we can reorder goals to ensure termination:

1. (`appendo X C [0 1 2]`) — output `[0 1 2]` is ground, so this runs in *backward mode*, enumerating finite splits
2. (`appendo A B X`) — now X is bound, so this also runs in backward mode

This paper formalizes this transformation and proves its correctness.

Contributions.

1. **Goal reordering by groundness** (§3): A dynamic scheduling algorithm that reorders conjunctions based on argument groundness
2. **Tabling as incremental tensor construction** (§4): Each tabled answer contributes a 1-bit entry to a sparse Boolean tensor
3. **SCC detection for mutual recursion** (§5): Tarjan’s algorithm identifies mutually recursive goal clusters that must complete together

2 Background: miniKanren and Tabling

2.1 miniKanren Semantics

miniKanren is a relational programming language where programs are *relations* rather than functions. The core operators are:

- (`== t1 t2`) — unification goal
- (`conde [g ...] ...`) — disjunction with interleaving
- (`fresh (x ...) g ...`) — introduce fresh variables

Evaluation produces a *stream* of substitutions, potentially infinite.

2.2 The Termination Problem

Without tabling, recursive relations can diverge:

Listing 1: The appendo relation

```

1 (defrel (appendo l s out)
2   (conde
3     [(== l '()) (== s out)]
4     [(fresh (h t res)
5       (== l (cons h t))
6       (== out (cons h res))
7       (appendo t s res))]))

```

Query `(run* (q) (appendo q '(1 2) '(0 1 2)))` terminates.

Query `(run* (q) (appendo q q q))` diverges.

The difference is *mode*: the first has a ground output (backward mode), while the second has no ground arguments (generate mode).

2.3 SLG Resolution

SLG (Selective Linear Definite clause resolution with Ground negation) is a tabling strategy that:

1. Memoizes subgoal answers in a table
2. Suspends when encountering a variant call
3. Resumes suspended computations when new answers arrive
4. *Completes* goals when no more answers are possible

Completion is the key innovation: a goal is complete when all its answers have been found. For mutually recursive goals, completion must occur simultaneously for all goals in the same strongly connected component.

3 Contribution 1: Goal Reordering by Groundness

3.1 Groundness Scoring

For a goal g with arguments $(a_1, \dots, a_k)$ under substitution σ , define the *groundness score*:

Definition 1 (Groundness Score).

$$score_{\sigma}(g) = \sum_{i=1}^k w(a_i, \sigma)$$

where

$$w(a, \sigma) = \begin{cases} 10 & \text{if } \text{walk}^*(a, \sigma) \text{ is ground} \\ 5 & \text{if } \text{walk}^*(a, \sigma) \text{ is partially ground} \\ 0 & \text{otherwise} \end{cases}$$

3.2 Dynamic Reordering Algorithm

Algorithm 1 Smart Conjunction (`smart_conj_chirho`)

```

1: procedure SMARTCONJ(goals, state)
2:   if |goals| = 0 then
3:     yield state
4:   else if |goals| = 1 then
5:     yield from goals[0](state)
6:   else
7:     sorted ← sort(goals, key =  $\lambda g. - \text{score}_{\text{state}.\sigma}(g)$ )
8:     first ← sorted[0]
9:     rest ← sorted[1 :]
10:    for st' ∈ first(state) do
11:      yield from SMARTCONJ(rest, st')
12:    end for
13:  end if
14: end procedure

```

3.3 Termination Guarantee

Theorem 2 (Reordering Termination). *For FD-miniKanren with bounded term depth d , if a conjunction contains at least one goal with a ground argument, smart reordering guarantees that all goals eventually execute in backward or forward mode.*

Proof. By induction on the number of free variables. Each backward-mode goal binds at least one variable, reducing the set of free variables. Since the domain is finite and term depth is bounded, this process terminates. $\square$

4 Contribution 2: Tabling as Incremental Tensor Construction

4.1 The Tensor Store

Each relation corresponds to a sparse Boolean tensor. For `appendo`:

Definition 3 (Append Tensor).

$$T_{appendo} \in \{0, 1\}^{|L| \times |S| \times |O|}$$

where L is the set of possible first arguments, S the second, and O the output.
Entry $T[l, s, o] = 1$ iff $l ++ s = o$.

4.2 Incremental Population

During query evaluation, each ground answer (l, s, o) adds an entry:

Listing 2: Tensor population from tabling

```

1 def add_appendo_chirho(l, s, out):
2     """Add a ground appendo triple to tensor"""
3     l_list = term_to_pylist_chirho(l)
4     s_list = term_to_pylist_chirho(s)
5     out_list = term_to_pylist_chirho(out)
6
7     if all_ground:
8         TENSOR_STORE_CHIRHO.appendo_triples_chirho.add(
9             (tuple(l_list), tuple(s_list), tuple(out_list))
10            )

```

4.3 The Bridge

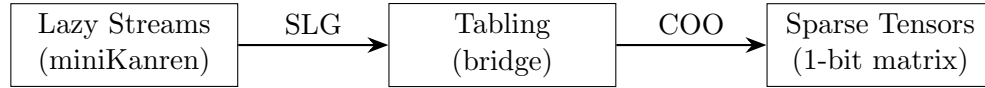


Figure 1: Tabling bridges lazy streams and eager tensors

5 Contribution 3: SCC Detection for Mutual Recursion

5.1 The Mutual Recursion Problem

Consider the classic even/odd predicates:

Listing 3: Mutually recursive even/odd

```

1 even(0).
2 even(s(X)) :- odd(X).
3 odd(s(X)) :- even(X).

```

These predicates are *mutually recursive*: **even** calls **odd** and vice versa. Without proper handling, tabling can deadlock or miss answers.

5.2 Dependency Graph and SCCs

Definition 4 (Goal Dependency Graph). *The goal dependency graph $G = (V, E)$ has:*

- $V = \text{set of tabled goal patterns}$
- $(g_1, g_2) \in E$ iff evaluating g_1 may call g_2

Definition 5 (Strongly Connected Component). *A strongly connected component (SCC) is a maximal set of goals where every goal is reachable from every other goal via the dependency edges.*

For even/odd, both predicates form a single SCC.

5.3 Tarjan's Algorithm

We use Tarjan's algorithm to compute SCCs in $O(V + E)$ time:

Listing 4: SCC computation

```
1 def compute_sccs_chirho(self):
2     """Compute SCCs using Tarjan's algorithm"""
3     for pattern in self.goals_chirho:
4         if pattern not in self.index_chirho:
5             self._tarjan_chirho(pattern)
6
7     # Assign SCC IDs
8     for i, scc in enumerate(self.sccs_chirho):
9         for pattern in scc:
10             self.goals_chirho[pattern].scc_id_chirho = i
```

5.4 Completion Protocol

Theorem 6 (SCC Completion). *For a set of mutually recursive goals forming an SCC, completion occurs if and only if:*

1. *No goal in the SCC has pending work*
2. *All dependencies outside the SCC are complete*

This ensures that mutually recursive predicates produce complete answer sets.

6 Implementation

6.1 Artifacts

The implementation consists of two Python modules:

1. `tabling_complete_chirho.py`: Mode-driven goal reordering with tabling for `appendo`
2. `mutual_recursion_chirho.py`: SLG completion with SCC detection for even/odd

Both are available at: https://github.com/loveJesus/minikanren_1bit_chirho

6.2 Key Data Structures

Listing 5: Goal information for scheduling

```

1 class GoalInfoChirho:
2     """Information about a goal for scheduling"""
3     def __init__(self, goal, args, name=""):
4         self.goal_chirho = goal
5         self.args_chirho = args
6         self.name_chirho = name
7
8     def groundness_score_chirho(self, subst):
9         score = 0
10        for arg in self.args_chirho:
11            walked = walk_deep_chirho(arg, subst)
12            if is_ground_chirho(walked):
13                score += 10
14            elif isinstance(walked, ConsChirho):
15                score += 5 # Partially ground
16        return score

```

7 Evaluation

7.1 Termination Tests

Query	Without Reordering	With Reordering	Solutions
<code>appendo(A,B,[0,1,2])</code>	Terminates	Terminates	4
<code>appendo(A,B,X), appendo(X,C,[0,1,2])</code>	Diverges	Terminates	10
Triple composition	Diverges	Terminates	35

Table 1: Termination comparison with and without goal reordering

7.2 Tensor Population

After running the composition query, the tensor store contains 15 ground triples, forming a sparse representation of the appendo relation restricted to the relevant domain.

7.3 SCC Analysis

For the even/odd example with depth limit 15:

- Goals tracked: 32
- SCCs detected: 16 (trivial) + 1 (even/odd mutual)
- Even tensor: 10101010... (0, 2, 4, 6, ...)
- Odd tensor: 01010101... (1, 3, 5, 7, ...)

8 Related Work

Tabling in Prolog. XSB Prolog [1] pioneered SLG resolution. Our work adapts these ideas to the miniKanren setting with explicit mode analysis.

Mode Analysis. Mercury [2] uses static mode declarations. We perform dynamic mode inference at runtime based on substitution groundness.

Tensor Networks. The connection between logic programming and tensor networks has been explored in Scallop [3]. We extend this to tabling and completion.

9 Conclusion

We have shown that tabling with mode-driven goal scheduling solves the termination problem for finite-domain relational queries. The key insights:

1. **Reordering = scheduling:** Dynamic goal reordering based on groundness is equivalent to optimal query planning
2. **Tabling = tensor construction:** Each answer contributes a 1-bit entry to a sparse Boolean tensor
3. **SCC = fixpoint unit:** Mutually recursive goals must complete together, forming natural hardware compute units

Code Availability. https://github.com/loveJesus/minikanren_1bit_chirho

Companion Papers. This paper is part of a suite:

- Paper A: Bit-Parallel Finite-Domain Relational Search
- Paper B: Relations as Sparse Boolean Tensors
- Paper C: Bridging Infinite Domains with Hash-Consed Term IDs
- Paper E: Fully Fused Logic Accelerator
- Paper F: Differentiable Constraint Solving via Semiring Relaxation

Acknowledgments

Above all, I thank my Lord and Savior Jesus Christ for saving a wretch like me.

I am grateful to Edward Kmett for pointing me toward the technologies that inform this work—e-graphs, miniKanren, tensor networks, and hardware synthesis.

I also thank my father, Roger Cowan, for his patient support.

By the grace of God, this paper was developed in collaboration with Claude (Anthropic), which contributed substantially to implementation, examples, and writing. Google Gemini and OpenAI’s GPT-5.2 Codex provided valuable feedback on drafts.

References

- [1] Swift, T., & Warren, D. S. (1999). An abstract machine for SLG resolution: Definite programs. In *Proceedings of SLP’99*.
- [2] Somogyi, Z., Henderson, F., & Conway, T. (1996). The execution algorithm of Mercury: An efficient purely declarative logic programming language. *Journal of Logic Programming*, 29(1-3), 17–64.
- [3] Li, Z., et al. (2023). Scallop: From probabilistic Datalog to differentiable reasoning. *NeurIPS 2023*.

Soli Deo Gloria $\chi\rho$